



LABORATÓRIO DE SW BÁSICO

■ EMENTA

■ Laboratório de Software Básico CH: 60 h Créditos: 0.2.0

- Pré-requisito(s): Arquitetura de Computadores, Sistemas Operacionais I Ementa. Linguagens de máquina e linguagem de montagem ("Assembly"). Montadores, macroprocessadores, carregadores e ligadores. Organização de memória em um programa; área estática, área de alocação dinâmica, registros de ativação. Ligação e relocação de programas objeto. Depuração de código Assembly. Nível de máquina de sistemas operacionais. Serviços e chamadas ao Sistema Operacional.



LABORATÓRIO DE SW BÁSICO

■ Bibliografia Básica

- ZHIRKOV, I. Programação em Baixo Nível: C, Assembly e execução de programas na arquitetura Intel 64. Novatec Editora, 2018.
- HENNESSY, J. L. e PATTERSON, D. A. Organização e Projeto de Computadores: A Interface Hardware/Software. Elsevier Academic, 2014.
- KERRISK, M. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press, 2010.



LABORATÓRIO DE SW BÁSICO

- **Bibliografia Complementar**

- VAN DER LINDEN, P. Expert C Programming: Deep Secrets. Prentice Hall, 1994.
- HYDE, R. Write Great Code, Volume 1: Understanding the Machine. No Starch Press, 2004.
- HYDE, R. Write Great Code, Volume 2: Thinking Low-Level, Writing High-Level. No Starch Press, 2006.



Introdução

- **Relação Binário e Hexadecimal:**

Quando os zeros e uns de quatro bits binários são agrupados (de 0000 a 1111; chamado *nibble*), eles podem ser representados por um simples dígito Hexadecimal (de 0 a F).

- Oito bits binários (representam qualquer valor decimal de 0 a 255) são representados por dois dígitos Hexadecimal. Dois dígitos Hexadecimais (8-bit binários) são chamados de **bytes**. O maior valor decimal que pode ser representado por 16 bits é 65.535 que é equivalente a FFFF em hexadecimal.



ARQUITETURA 8086

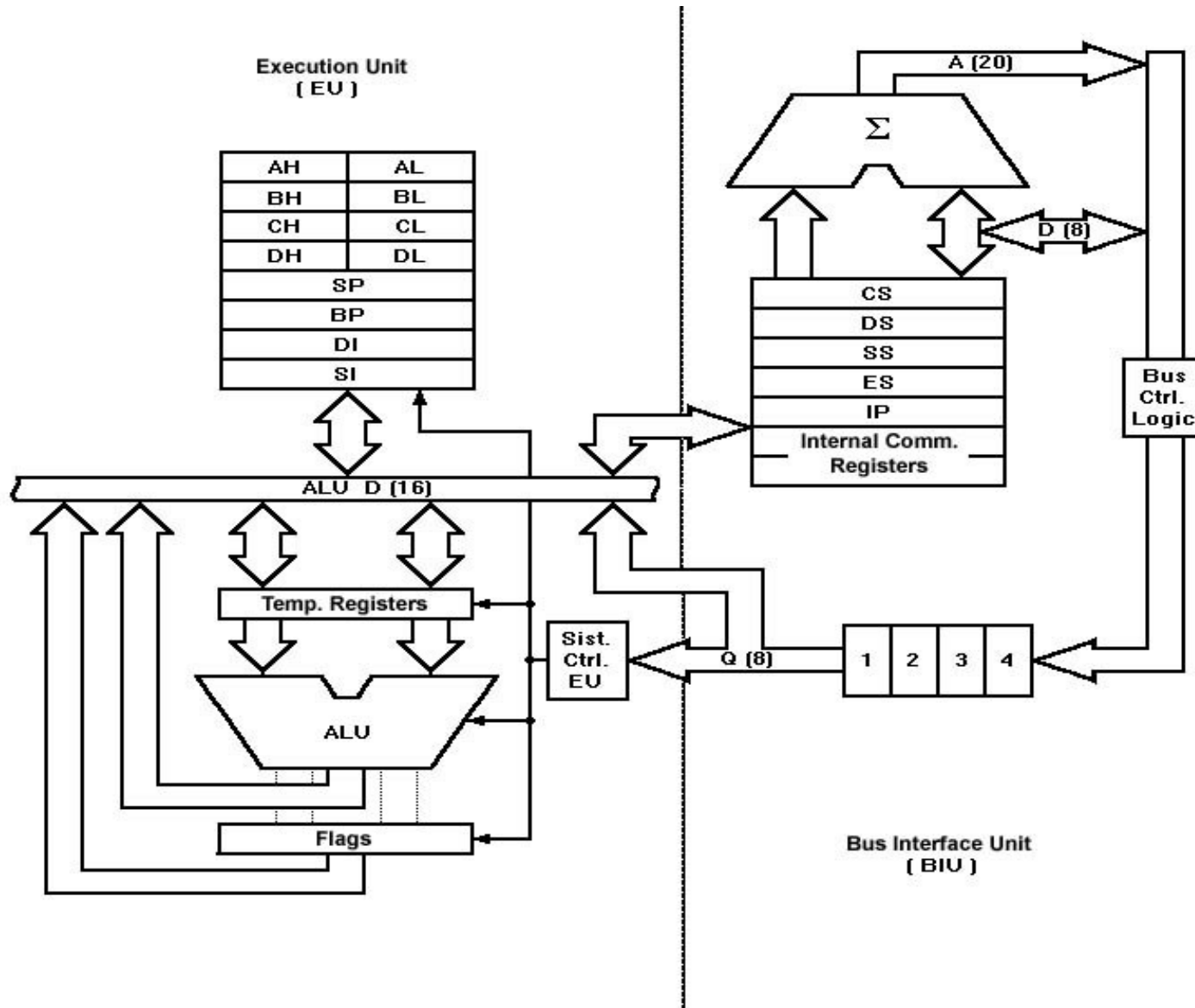
- Microprocessador cuja arquitetura está na base de todos os processadores da série 80x86.
- Qualquer processador Intel do 80186 ao Pentium III é retro compatível com o 8086, podendo trabalhar como se fosse um 8086.
- 8086 é o ponto de partida ideal para que se possa compreender o funcionamento de uma gama completa de microprocessadores.



ARQUITETURA 8086

- CPU com registradores de 16 bits.
- Barramento de dados de 16 bits.
- Barramento de endereços de 20 bits.
- Memória limitada a 1Mb.
- Dividida em segmentos de 64Kb.
- Bytes na memória não possuem endereço único.
- 85 instruções básicas.
- Sem cache, sem memória virtual.

ARQUITETURA INTERNA 8086





ARQUITETURA 8086

■ EXECUTION UNIT – EU

- ❑ Responsável pela decodificação e execução de todas as instruções de código.
- ❑ Constituída por uma ALU (Arithmetic Logic Unit), registrador flags, quatro registradores de uso geral, quatro registradores offset para acesso à memória e lógica de controle de queue.
- ❑ Processa as instruções no registrador de queue da BIU, processa a decodificação destas instruções, gera endereços de operandos - se necessário, transfere estes endereços para a BIU, requisitando ciclos de leitura/escrita na memória ou I/O e processa a operação especificada pela instrução sobre os operandos.



ARQUITETURA 8086

■ EXECUTION UNIT – EU

- ❑ Testa os flags de estado e controle, alterando-os se necessário conforme o resultado da instrução corrente.
- ❑ Quando a EU executa uma instrução de salto, ou desvio, ela transfere o conteúdo para uma nova posição de memória, neste instante, a BIU reinicializa a queue passando a executar a pre-fetch a partir da nova localização de memória.



ARQUITETURA 8086

■ BUS INTERFACE UNIT – BIU

- ❑ Responsável pelas funções de busca e colocação em queue de instruções, leitura e gravação de operandos e realocação de endereços. Esta unidade trata também do controle do barramento.
- ❑ Para realizar estas funções, a BIU possui: registradores e segmentos, registradores de comunicação interna, apontador de instrução, queue de registradores, somador de endereços e lógica de controle do barramento.

ARQUITETURA 8086

■ BUS INTERFACE UNIT – BIU

- Utiliza o mecanismo chamado fluxo de instrução para implementar a arquitetura pipeline. O registrador queue permite que haja uma pre-fetch de até 6 bytes de código de instrução (4 bytes no caso do 8088). Sempre que a queue tenha 2 bytes livres, e a EU não esteja a executar operações de escrita ou leitura em memória, a BIU irá efetuar uma operação de pre-fetch. Como o barramento de dados é de 16 bits, a BIU efetua um pre-fetch de 2 bytes por ciclo.

ARQUITETURA 8086

■ BUS INTERFACE UNIT – BIU

- Quando o registrador queue está completo, e a EU não está a executar operações de escrita/leitura na memória, a BIU não efetua ciclos de barramento. Estes tempos por inatividade das vias são chamados *Idle States*.
- Quando a BIU está a executar uma pre-fetch, e a EU a executar operações de escrita/leitura da memória ou I/O, a BIU completa primeiro a pre-fetch e só depois atenderá o pedido da EU.

Estas duas unidades interagem diretamente, mas normalmente funcionam assincronamente como dois processadores isolados.

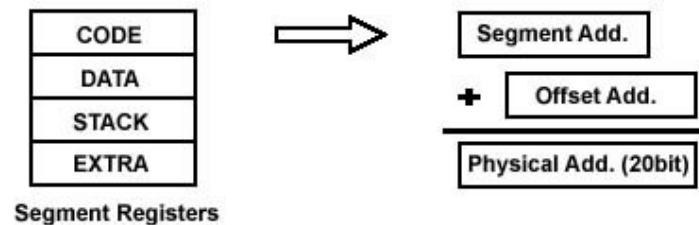
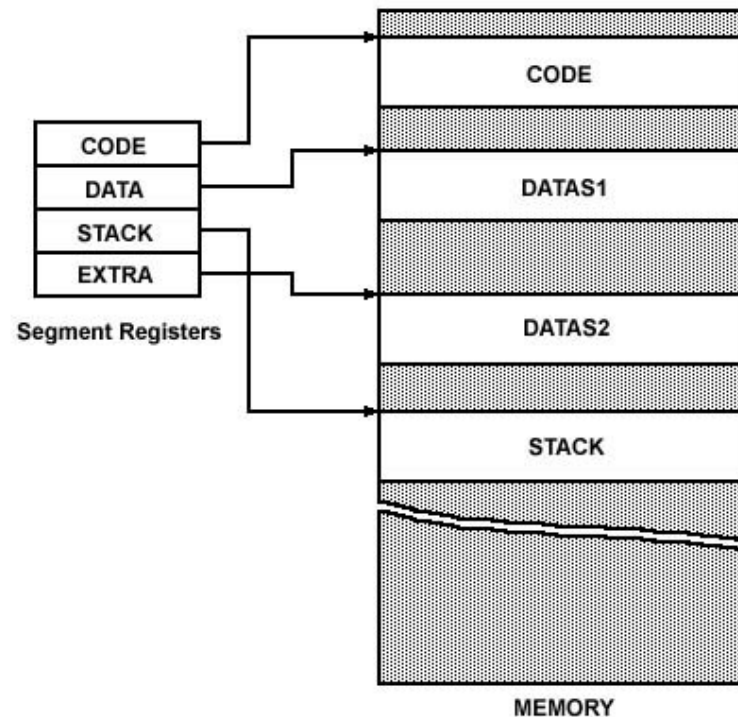
ARQUITETURA 8086

■ ORGANIZAÇÃO DA MEMÓRIA

- ❑ Cada byte na memória possui um endereço de 20 bits iniciando em 0 até $2^{20}-1$ ou seja, 1M de memória endereçável;
- ❑ Endereços são representados por 5 dígitos hexadecimais; de 00000 - FFFFF
- ❑ Problema: 20 bits de endereços é grande demais para ser colocado em registradores de 16 bits;
- ❑ Solução: Segmentação de memória
 - Blocos de memória de 64K consecutivos (65.536);
 - Um número de segmento é um número de 16 bits;
 - Faixa de um endereços de um segmento vai de 0000 a FFFF
 - Em um segmento, uma posição de memória em particular é especificado como sendo um offset (deslocamento);
 - Um offset também tem faixa de 0000 a FFFF

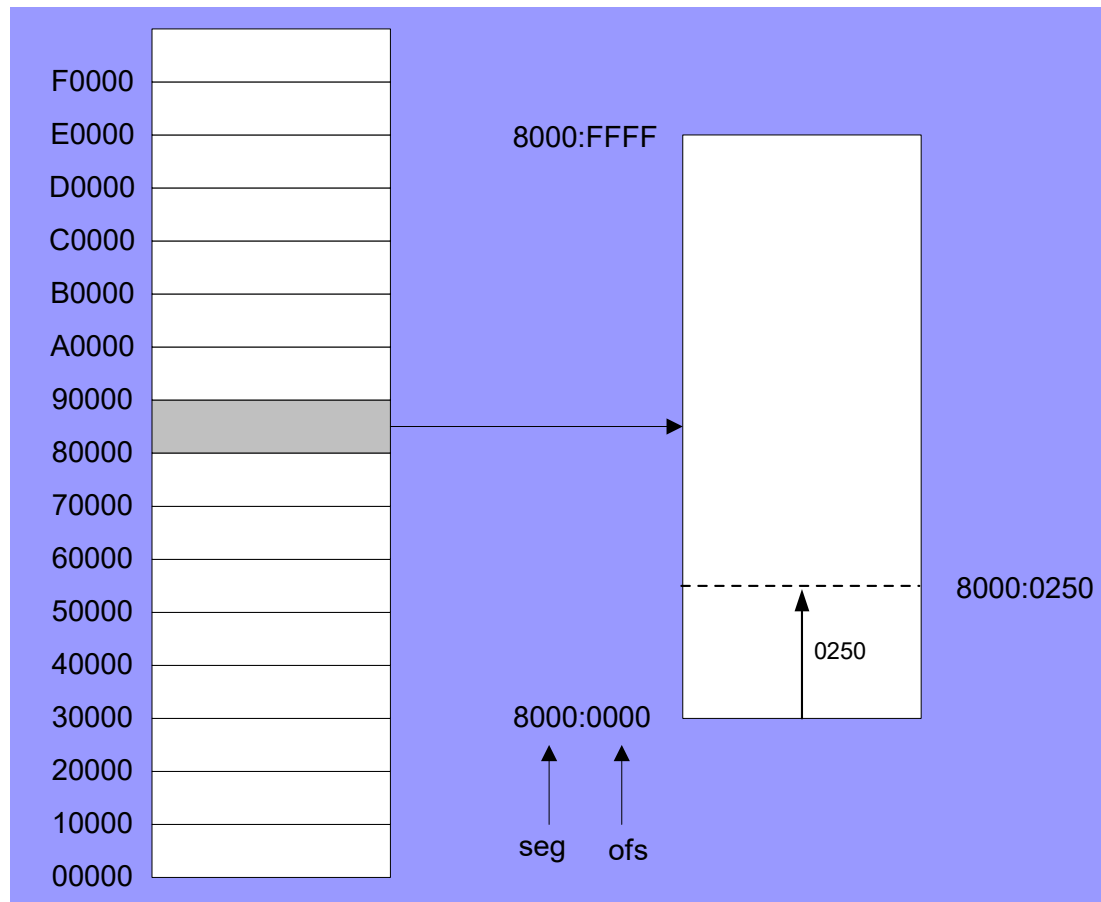
ARQUITETURA 8086

■ CARACTERÍSTICAS DA MEMÓRIA



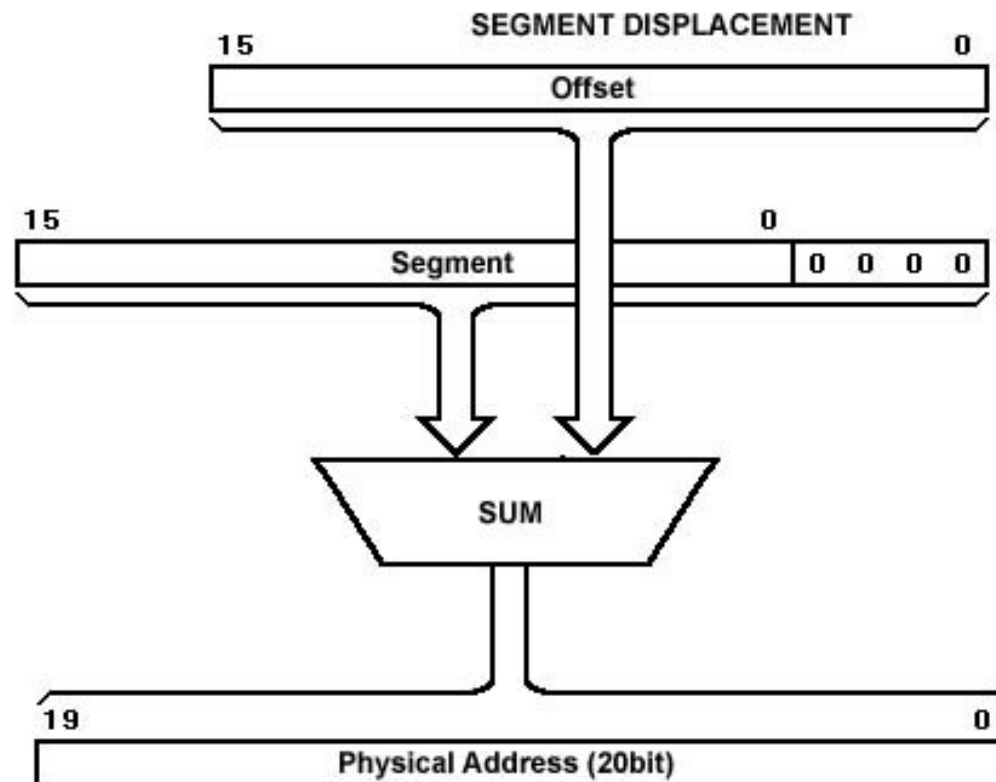
ARQUITETURA 8086

■ SEGMENTAÇÃO DA MEMÓRIA



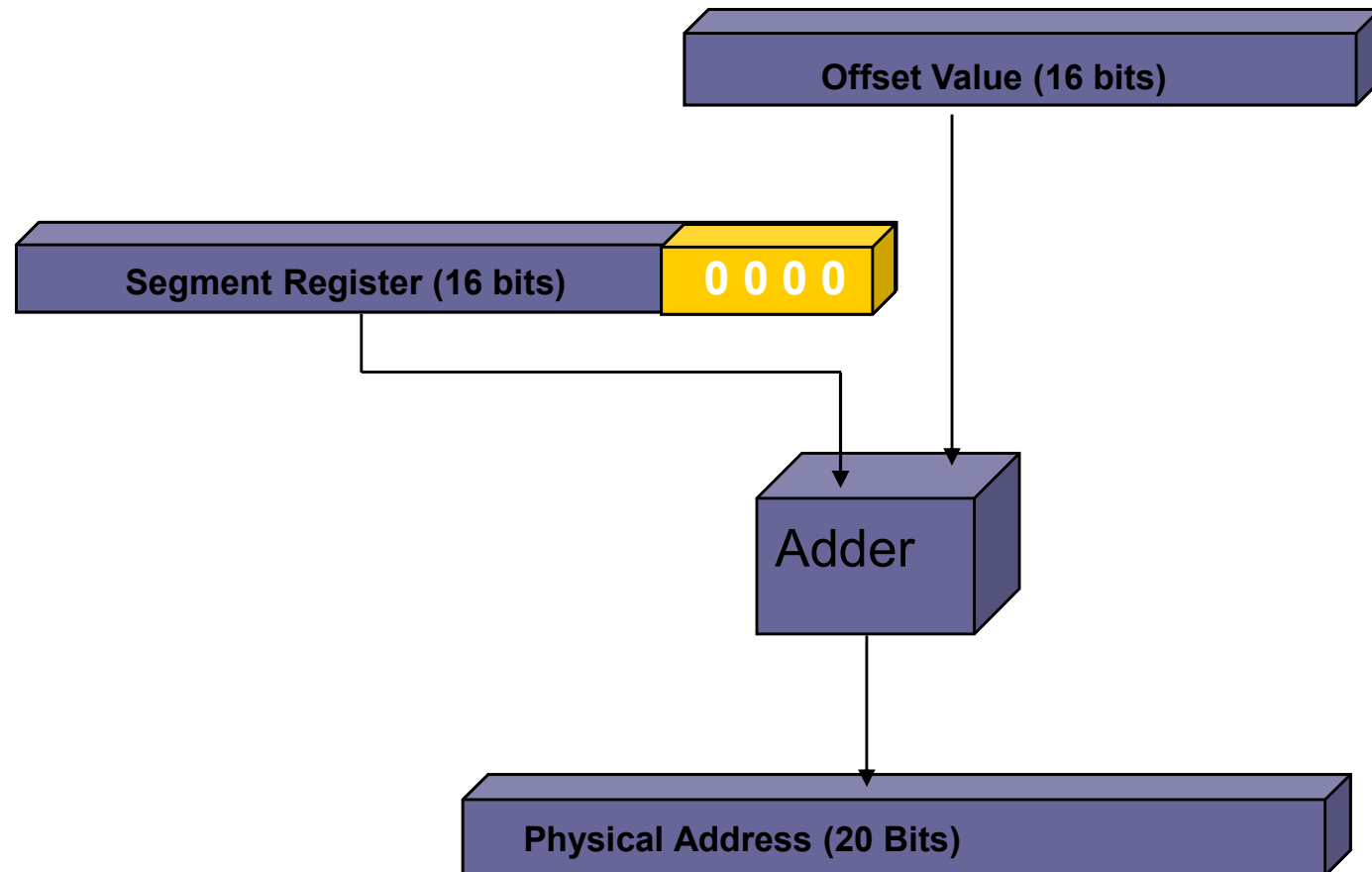
ARQUITETURA 8086

■ GERAÇÃO DO ENDEREÇO FÍSICO



ARQUITETURA 8086

- Registrador de segmento * 16 + offset





ARQUITETURA 8086

- **Possui 14 registradores de 16 bits sendo:**

- ☐ 4 de propósito geral
- ☐ 4 registradores de segmento para acesso à memória
- ☐ 1 apontador de instruções
- ☐ 1 registrador de flags

ARQUITETURA 8086

■ Registradores de propósito geral:

REG. 16 BITS	PROPÓSITO	REGS. 8 BITS	
AX	ACUMULADOR	AH	AL
BX	BASE	BH	BL
CX	CONTADOR	CH	CL
DX	DADOS	DH	DL

ARQUITETURA 8086

- Registradores offset para acesso à memória:

REG. 16 BITS	PROPÓSITO
SP	STACK POINTER
BP	BASE POINTER
SI	SOURCE INDEX
DI	DESTINATION INDEX

ARQUITETURA 8086

- Registradores de segmento para acesso à memória:

REG. 16 BITS	PROPÓSITO
CS	SEGMENTO DE CÓDIGO
DS	SEGMENTO DE DADOS
SS	SEGMENTO DE PILHA
ES	SEGMENTO EXTRA

ARQUITETURA 8086

■ Registrador apontador de instruções:

REG. 16 BITS	PROPÓSITO
IP	APONTADOR DE INSTRUÇÕES

Aponta sempre para próxima instrução que será executada. É incrementado da quantidade de bytes da instrução que foi previamente executada.

Debug

■ Instalação em versões do Windows 7:

- ☐ Instalar o DOSBOX.EXE
- ☐ Copiar o DEBUG.COM em um diretório C:\
- ☐ Inicializar o DOSBOX.EXE a partir do desktop
- ☐ Digite MOUNT c C:\DOSBOX <enter>
- ☐ Digite C:\ <enter>
- ☐ Digite debug <enter>

Agora você poderá usar o debug

Debug

- Programa incluso no DOS que permite ao programador monitorar a execução de um programa para fins de depuração.
- Usando o Debug:
 - Debug Enter
C:>DEBUG<enter>
-Prompt do Debug
 - Debug Exit
-Q<enter>
C:>

Debug

■ Mostrando os registradores

-R<enter>

AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000

DS=0D00 ES=0D00 SS=0D00 CS=0D00 IP=0100 NV UP DI PL NZ NA PO NC

0D00:0100 B80100 MOV AX,0001

■ Modificando os registradores

-R CX <enter>

CX 0000

:0009<enter>

-R CX<enter>

CX 0009

:<enter>

-

Debug

- **Comando Assemble (A)** – permite ao programador digitar instruções em linguagem de montagem na memória.

-A 100<enter>

0B3C:0100 MOV AX,1<enter>

0B3C:0103 MOV BX,2<enter>

0B3C:0106 ADD AX,BX<enter>

0B3C:0108 INT 3<enter>

0B3C:0109<enter>

-

Debug

- **Comando Unassemble (U) – permite ao programador verificar o código de máquina na memória por meio das suas instruções em linguagem de montagem.**

```
-U 100 <enter>
```

```
0B3C:0100 B80100 MOV     AX,1
```

```
-U 100 103
```

```
0B3C:0100 B80100 MOV     AX,1
```

```
0B3C:0103 BB0200 MOV     BX,2
```

```
-
```

Debug

- **Comando Go (G) – permite ao programador executar instruções encontradas entre dois endereços.**

-G=100 108<enter>

AX=0004 BX=0003 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000

DS=0B3C ES=0B3C SS=0B3C CS=0B3C IP=0108 NV UP EI PL NZ NA PO NC

0B3C:0108 CC INT 3

Debug

- **Comando Proceed (P)** – permite ao programador através da execução de uma ou mais instruções de um programa verificar o efeito do programa nos registradores e/ou dados. Não executa as interrupções comando a comando.

-P=100 2<enter>

**AX=0001 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B3C ES=0B3C SS=0B3C CS=0B3C IP=0103 NV UP EI PL NZ NA PO NC
0B3C:0103 BB0200 MOV BX,0002**

**AX=0001 BX=0003 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B3C ES=0B3C SS=0B3C CS=0B3C IP=0106 NV UP EI PL NZ NA PO NC
0B3C:0106 01D8 ADD AX,BX**

-

Debug

- **Comando Trace (T)** – permite ao programador através da execução de uma ou mais instruções de um programa verificar o efeito do programa nos registradores e/ou dados.

```
-T=100 2<enter>
```

```
AX=0001 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000  
DS=0B3C ES=0B3C SS=0B3C CS=0B3C IP=0103 NV UP EI PL NZ NA PO NC  
0B3C:0103 BB0200 MOV BX,0002
```

```
AX=0001 BX=0003 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000  
DS=0B3C ES=0B3C SS=0B3C CS=0B3C IP=0106 NV UP EI PL NZ NA PO NC  
0B3C:0106 01D8 ADD AX,BX
```

-

Debug

- **Comando Dump (D) – permite ao programador examinar o conteúdo da memória.**
- **Comando Fill (F) - permite ao programador preencher a memória com dados.**

```
-F 100 LF 00<enter>
```

```
-D 100 LF
```

```
0B3C:0100 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

```
-F 110 11F 20
```

```
-D 100 11F
```

```
0B3C:0100 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

```
0B3C:0110 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20
```

```
-F 120 LF 20
```

```
-
```

Debug

- **Comando Edit (E) - permite ao programador modificar o conteúdo da memória.**

-E 120 'John Smith'

-D 120 LF

0B3C:0120 4A 6F 68 6E 20 53 6E 69 74 68 20 20 20 20 20 20 John Smith

Debug

- **A carga de programas de um arquivo específico requer dois comandos, o comando Name, N, e o comando Load, L.**

-N C:\PROG1.EXE

-L

- **Carregando programas enquanto carrega o Debug.**

C:\DEBUG C:\PROG1.EXE

Debug

■ Links para sites úteis:

□ **DEBUG/ASSEMBLY TUTORIAL by Fran Golden**

■ <http://www.datainstitute.com/debug1.htm>

□ **Rough Guide to Assembly**

■ <http://www.geocities.com/riskyfriends/prog.html>

□ **Paul Hsieh's x86 Assembly Language Page**

■ <http://www.azillionmonkeys.com/qed/asm.html>

ARQUITETURA 8086

■ Registrador de flags:

Registrador de 16 bits sendo que apenas 09 são usados. Destes 09, **06 são de estado** (reportam o resultado da última operação lógica ou aritmética executada pela ULA e **03 são de controle** (servem para instruir como a CPU deverá operar).

REGISTRADOR DE FLAGS															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
U	U	U	U	OF	DF	IF	TF	SF	ZF	U	AF	U	PF	U	CF

U - undefined

ARQUITETURA 8086

■ Registrador de flags:

□ Carry Flag (CF) – BIT 0

CF=1 – quando em operações de adição ocorrer vai um no MSB ou ocorrer empréstimo no MSB na subtração. Sinalização no DEBUG – **CY** (**C**arry **Y**es).

CF=0 – quando em operações de adição não ocorrer vai um no MSB ou não ocorrer empréstimo no MSB na subtração. Sinalização no DEBUG – **NC** (**N**o **C**arry).

MSB – MOST SIGNIFICANT BIT

ARQUITETURA 8086

■ Registrador de flags:

□ Parity Flag (PF) – BIT 2

PF=1 – quando em operações lógicas ou aritméticas ocorrer um número par de bits em 1 no byte inferior do resultado. Sinalização no DEBUG – **PE** (**P**arity **E**ven).

PF=0 – quando em operações lógicas ou aritméticas ocorrer um número ímpar de bits em 1 no byte inferior do resultado. Sinalização no DEBUG – **PO** (**P**arity **O**dd).

EVEN – PAR

ODD - ÍMPAR

ARQUITETURA 8086

■ Registrador de flags:

□ Auxiliary Flag (AF) – BIT 4

AF=1 – quando em operações de adição ocorrer vai um do bit 3 para o bit 4 ou ocorrer empréstimo nesse mesmo bit na subtração. Sinalização no DEBUG – **AC** (**A**uxiliary **C**arry).

AF=0 – quando em operações de adição não ocorrer vai um do bit 3 para o bit 4 ou não ocorrer empréstimo nesse mesmo bit na subtração. Sinalização no DEBUG – **NA** (**N**ot **A**uxiliary **C**arry).

ARQUITETURA 8086

■ Registrador de flags:

□ Zero Flag (ZF) – BIT 6

ZF=1 – quando em operações lógicas ou aritméticas ocorrer zero no resultado. Sinalização no DEBUG – **ZR (ZeRo)**.

ZF=0 – quando em operações lógicas ou aritméticas não ocorrer zero no resultado. Sinalização no DEBUG – **NZ (Not Zero)**.

ARQUITETURA 8086

■ Registrador de flags:

□ Sign Flag (SF) – BIT 7

SF=1 – quando em operações aritméticas sinalizadas o resultado for negativo. Sinalização no DEBUG – **NG** (**NeG**ative).

SF=0 – quando em operações aritméticas sinalizadas o resultado for positivo. Sinalização no DEBUG – **PL** (**PL**us).

O MSB do resultado de uma operação aritmética é copiado no SF. Esse flag só é válido se não ocorrer overflow.

ARQUITETURA 8086

■ Registrador de flags:

□ Overflow Flag (OF) – BIT 11

OF=1 – quando em operações aritméticas sinalizadas o resultado sinalizado não couber no destino.

Sinalização no DEBUG – **OV** (**OV**erflow).

OF=0 – quando em operações aritméticas sinalizadas o resultado sinalizado couber no destino. Sinalização no DEBUG – **NV** (**N**o **O**verflow).

Quando ocorrer overflow o bit de sinal não reporta corretamente o sinal do resultado da operação processada.

ARQUITETURA 8086

■ Registrador de flags:

□ Trap Flag (TF) – BIT 8

TF=1 – coloca a CPU em modo único passo. Usado para depuração. Um comando é executado e a CPU espera um enter para executar o próximo. Não existe sinalização no DEBUG.

TF=0 – coloca a CPU em modo normal de execução. Não existe sinalização no DEBUG.

Não existe comando para modificar diretamente esse flag. As modificações somente poderão ser feitas via lógica.

ARQUITETURA 8086

■ Registrador de flags:

□ Interruption Flag (IF) – BIT 9

IF=1 – habilita interrupções externas do tipo INT no pino INTR da CPU. Sinalização no DEBUG **EI** (**E**nable **I**nterrupt).

IF=0 – desabilita interrupções externas do tipo INT no pino INTR da CPU. As interrupções solicitadas ficam pendentes, isto é são mascaradas. Sinalização no DEBUG **DI** (**D**isable **I**nterrupt).

Quando a CPU estiver executando uma rotina crítica e não puder ser interrompida o flag IF deve ser resetado. Nessa situação as interrupções ficam pendentes.

CLI -> IF=0 STI -> IF =1

ARQUITETURA 8086

■ Registrador de flags:

□ Direction Flag (DF) – BIT 10

DF=1 – coloca a CPU no modo auto decremento. Após cada operação de movimentação de strings os registradores SI e DI são decrementados da quantidade de bytes que foi movida na memória. Sinalização no DEBUG **DN (DowN)**.

DF=0 – coloca a CPU no modo auto incremento. Após cada operação de movimentação de strings os registradores SI e DI são incrementados da quantidade de bytes que foi movida na memória. Sinalização no DEBUG **UP (UP)**.

CLD -> DF=0 STD -> DF =1



FUNCIONAMENTO DOS FLAGS

- **Carry Flag (CF) – BIT 0**

Linguagem Assembly

Programa

- **Série de declarações que são instruções da linguagem assembly ou diretivas.**
 - **Instruções são declarações tais como `ADD AX,BX` que são traduzidas em código de máquina.**
 - **Diretivas ou pseudo-comandos são declarações usadas pelo programador para direcionar o montador como proceder. Não geram código em linguagem de máquina.**

Linguagem Assembly

■ Formato declaração:

- [label:] operação [operandos][;comentários]

■ Label:

- Não pode exceder 31 caracteres.
- Consiste em:
 - Caracteres alfabéticos maiúsculos e minúsculos.
 - Dígitos 0 a 9.
 - Caracteres Especiais (?), (.), (@), (_), and (\$).
- O primeiro caracter não pode ser um dígito.
- O ponto pode ser somente usado como o primeiro caracter, mas não é recomendado.
Várias palavras reservadas começam com ponto em versões posteriores do MASM.

Linguagem Assembly

Programa

■ Label:

- ☐ Deve terminar com dois pontos quando ele se refere a um código de operação que gera instrução.
- ☐ Não precisa terminar com dois pontos quando se refere a uma diretiva.

■ Mnemônicos (Operações) e operandos:

- ☐ Instruções são traduzidas para código de máquina.
- ☐ Diretivas não geram código de máquina. Usadas pelo montador para organizar o programa e direcionar o processo de montagem.

Linguagem Assembly

Programa

■ Comentários:

- Começam com um “;”.
- Ignorados pelo montador.
- Pode estar em uma linha por si só ou no final de uma linha:
 - ;Meu primeiro comentário
 - MOV AX,1234H ;Inicializando....
- Indispensável aos programadores porque eles tornam o entendimento mais fácil para alguém que leia o programa.

Constantes

- EQU é usado para definir constantes ou assinalar nomes a expressões.
- Forma:
 - Nome EQU expressão.
- Exemplos:
 - `PI EQU 3.1415`
 - `Raio EQU 25`
 - `Circunferencia EQU 2*PI*Raio`

Variáveis

- DB - define byte.
- DW - define word.
- DD – define double word.
- DT – define ten bytes
- DQ – define quadword
- Forma:
 - Diretiva de Variável oper, . . ,oper
- Exemplos:
 - Alpha db 'ABCDE'
 - Alpha2 db 'A','B','C','D','E'
 - Alpha3 db 41h,42h,43h,44h,45h
 - Word1 dw 3344h
 - Double_word1 dd 44332211h

Estrutura de uma Programa .COM

Codigo SEGMENT

Assume CS:Codigo;DS:Codigo; ES:Codigo; SS:Codigo

Org 100H

Entrada: JMP Nomeprog

DADOS AQUI

Nomeprog PROC NEAR

PROGRAMA AQUI

INT 20H

Nomeprog ENDP

Codigo ENDS

END Entrada

■ Diretivas

□ SEGMENT e ENDS.

Delimitam um segmento lógico do programa.

□ ASSUME

Informa ao montador qual dos segmentos definidos será indicado por qual registrador.

□ ORG 100H

Informa onde o IP será posicionado. Neste caso IP = 100H. PSP vai de CS:0000 até CS:00FF que equivale a 256 bytes.

□ Entrada: Label

Label definido para que o programa possa acessar a posição indicada por ele.



■ Diretivas

□ PROC e ENDP.

Delimitam um procedimento que pode ser do tipo NEAR ou FAR.

□ END

Informa ao montador que deve encerrar a montagem. O label entrada que segue informa que o processador deve entrar no programa nesse label que contém a primeira instrução a ser executada.

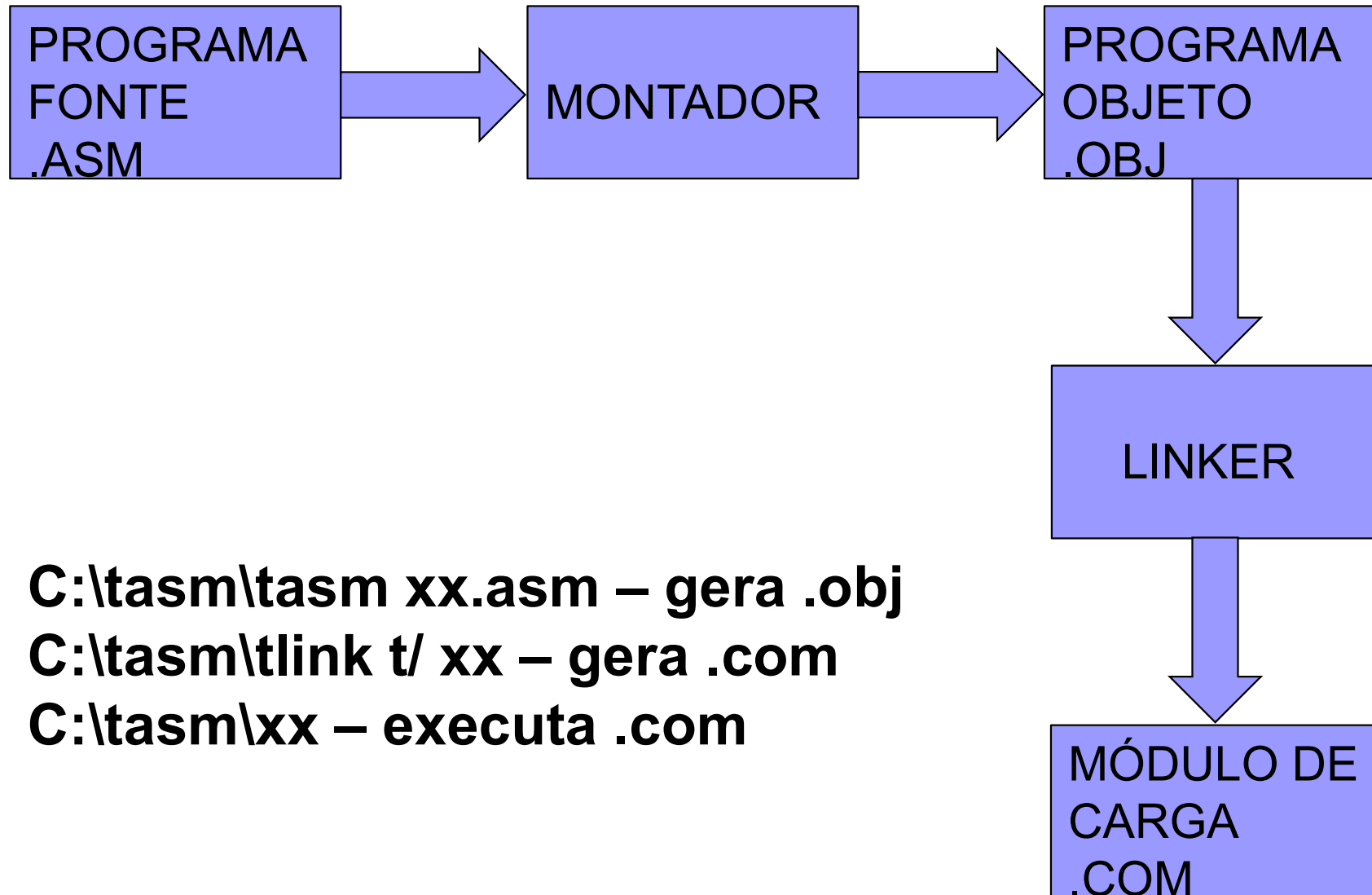


Editores de Texto

- Podem ser usados os seguintes editores para escrever programas.
 - ☐ Notepad (Windows).
 - ☐ Edit (DOS).
 - ☐ Qualquer outro editor capaz de gerar arquivos ASCII.

Processo de Montagem

■ Tradução



Interrupções do DOS e BIOS

- **Interrupções do DOS e BIOS são usadas para executar algumas funções muito úteis tais como exibição de dados no monitor, leitura de dados via teclado, etc.**
- **Elas são usadas pela identificação da opção do tipo de interrupção que é o valor armazenado no registrador AH fornecendo-se alguma informação extra quando a opção específica requerer.**

Interrupção 21H - DOS

- **Função 1 – permite entrar um simples caracter via teclado e ecoa o dado na tela.**
- **Registradores usados:**
 - **AH = 1**
 - **AL = código ASCII da tecla pressionada.**
- **Ex:**
 - **MOV AH,1**
 - **INT 21H**

Interrupção 21H - DOS

- **Função 2 – exibe um caracter simples na tela.**
- **Registradores usados:**
 - **AH = 2**
 - **DL = contém o código ASCII do caracter a ser exibido.**
- **Ex:**
 - **MOV AH,2**
 - **MOV DL,'A'**
 - **INT 21H**



Codigo SEGMENT

Assume CS: Codigo;DS:Codigo; ES: Codigo; SS:Codigo

Org 100H

Entrada: JMP Nomeprog

Nomeprog PROC NEAR

CALL Lertecia

MOV CL,AL

CALL Lertecia

MOV BL,AL

ADD CL,BL

MOV AL,CL

MOV AH,0

AAA

ADD AX,3030H

MOV BX,AX

MOV DL,BH

CALL Mostrarchar

MOV DL,BL

CALL Mostrarchar

INT 20H

Nomeprog ENDP



Lertecia PROC NEAR

MOV AH,01H

INT 21H

RET

Lertecia ENDP

Mostrarchar PROC NEAR

MOV AH,02H

INT 21H

RET

Mostrarchar ENDP

Codigo ENDS

END Entrada

Interrupção 21H - DOS

- **Função 8 – lê uma tecla sem ecoar.**
- **Registradores usados:**
 - **AH = 8**
 - **AL = Código ASCII da tecla pressionada.**
 - **Checa por ^C ou ^Break.**
- **Ex:**
 - **MOV AH,08**
 - **INT 21H**

Interrupção 21H - DOS

- **Função 9 – exibe na tela uma string, terminada por um \$.**
- **Registradores usados:**
 - **AH = 9**
 - **DX = offset do endereço da string a ser mostrada.**
- **Ex:**
 - **MOV AH,09**
 - **MOV DX,OFFSET MESS1**
 - **INT 21H**

Interrupção 21H - DOS

- Função 0AH – entrada de string.
- Registradores usados:
 - ☐ AH = 0AH
 - ☐ [DS:DX] = tamanho do buffer.
 - ☐ Buffer preenchido em DS:DX ecoa para a tela
 - ☐ Checa por ^C ou ^Break.
- Ex:

		0	0	0	0	0DH
--	--	---	---	---	---	-----

 - ☐ VAR DB 4,?, 5DUP(0)
 - ☐ MOV DX,OFFSET VAR
 - ☐ MOV AH,0AH
 - ☐ INT 21H

Interrupção 21H - DOS

- **Função 4CH – termina um processo retronando o controle para o processo pai ou ao DOS.**
- **Registradores usados:**
 - **AH = 4CH**
 - **AL = código binário de retorno.**
- **Ex:**
 - **MOV AH,4CH**
 - **INT 21H**

Interrupção 10H - BIOS

- **Função 0H – Seta o modo de vídeo.**

- **Registradores usados:**

- **AH = 0H**

- **AL = Modo de Vídeo.**

- **3H - CGA Color texto de 80X25**

- **7H – Texto Monocromático de 80X25**

- **Ex:**

- **MOV AH,0**

- **MOV AL,7**

- **INT 10H**

Interrupção 10H - BIOS

- **Função 2H – Seta o cursor em uma posição específica.**
- **Registradores usados:**
 - ☐ **AH = 2H**
 - ☐ **BH = 0H seleciona Página 0.**
 - ☐ **DH = Posição da linha.**
 - ☐ **DL = Posição da coluna.**
- **Ex:**
 - ☐ **MOV AH,2**
 - ☐ **MOV BH,0**
 - ☐ **MOV DH,12**
 - ☐ **MOV DL,39**
 - ☐ **INT 10H**

Interrupção 10H - BIOS

- **Função 6H – Rola janela para cima.**
Esta interrupção é também usada para limpar a tela quando você seta $AL = 0$.
- **Registradores usados:**
 - ☐ **AH = 6H**
 - ☐ **AL = número de linhas a rolar.**
 - ☐ **BH = atributo da tela.**
 - ☐ **CH = coord. y do topo esquerdo.**
 - ☐ **CL = coord. x do topo esquerdo.**
 - ☐ **DH = coord. y do canto inferior direito.**
 - ☐ **DL = coord. x do canto inferior direito.**

Interrupção 10H - BIOS

- **Exemplo de limpeza da tela:**

- ☐ **MOV AH,6**
- ☐ **MOV AL,0**
- ☐ **MOV BH,7**
- ☐ **MOV CH,0**
- ☐ **MOV CL,0**
- ☐ **MOV DH,24**
- ☐ **MOV DL,79**
- ☐ **INT 10H**

- **O código acima pode ser reduzido usando os registradores AX, BX e DX para mover dados do tamanho word em lugar de byte.**

Interrupção 16H - BIOS

■ Serviço 0 – Leia tecla

- ☐ Retorna o código de varredura da tecla em AH e o ASCII do que foi digitado em AL.
- ☐ MOV AH,0
- ☐ INT 16H

■ Serviço 1 – examine o teclado e leia

- ☐ Retorna o código de varredura da tecla em AH e o ASCII do que foi digitado em AL se ZF=0. se ZF=1 no retorno o buffer está vazio
- ☐ MOV AH,1
- ☐ INT 16H

Estrutura de uma Programa .EXE

Dados_Seg SEGMENT

DADOS AQUI

Dados_Seg ENDS

Codigo SEGMENT

Assume CS: Codigo;SS:Pilha

Org 100H

Nomeprog PROC FAR

PUSH DX

XOR AX,AX

PUSH AX

MOV AX, Dados_Seg

MOV DS,AX

Assume DS:Dados_Seg

PROGRAMA AQUI

RET

Nomeprog ENDP

Codigo ENDS

Pilha SEGMENT PARA STACK 'STACK'

DB 100 DUP(?)

Pilha ENDS

END Nomeprog



Modos de endereçamento

- Há várias maneiras para o processador acessar dados:
 - ☐ Imediato.
 - ☐ Registrador.
 - ☐ Memória.
 - Direta.
 - Indireto via Registrador.
 - Relativo via Registrador.
 - Indexado por base.
 - Relativo indexado por base.



Imediato

- Acesso direto à EU.
- O endereço é parte de uma instrução.
- Útil em inicializações.
- **MOV AX,11110000B**
- **MOV CL, F1H**



Registrador

- **Acesso direto à EU.**
- **A maioria das instruções é compacta e de execução mais rápida.**
- **Operandos são codificados na instrução.**
- **MOV BX,DX**
- **MOV AL,CL**



Memória

- Nas operações de leitura e escrita de memória a EU passa um valor de ofsset, o EA endereço efetivo, para a BIU que então gera o endereço físico, PA.

Direto

$$EA = \{\text{operando}\}$$

$$PA = \{DS\} \times 16 + \{\text{operando}\}$$

- **É o mais simples dos modos de endereçamento à memória.**
- **Acessa variáveis simples.**
- **MOV AX,SUM**
- **MOV CL,COUNT+5**
- **MOV [500H],DX**

Indireto por Registrador

$$EA = \begin{Bmatrix} (BX) \\ (DI) \\ (SI) \end{Bmatrix}$$

$$PA = \{DS\} \times 16 + \begin{Bmatrix} (BX) \\ (DI) \\ (SI) \end{Bmatrix}$$

- MOV AX, [BX]
- MOV [DI],DX

Relativo por Registrador

$$EA = \left\{ \begin{array}{l} (BX) \\ (BP) \\ (DI) \\ (SI) \end{array} \right\} + \left\{ \begin{array}{l} 8 \text{ bit deslocamento} \\ 16 \text{ bit deslocamento} \end{array} \right\}$$

$$PA = \left\{ \begin{array}{l} DS \\ SS \\ DS \\ DS \end{array} \right\} \times 16 + \left\{ \begin{array}{l} (BX) \\ (BP) \\ (DI) \\ (SI) \end{array} \right\} + \left\{ \begin{array}{l} 8 \text{ bit deslocamento} \\ 16 \text{ bit deslocamento} \end{array} \right\}$$

- Acesso à matrizes unidimensionais.
- `MOV AX,ARRAY[BX]`
- `MOV MESSAGE[DI], DL`

Relativo por Base Indexado

$$EA = \left\{ \begin{matrix} (BX) \\ (BP) \end{matrix} \right\} + \left\{ \begin{matrix} (DI) \\ (SI) \end{matrix} \right\} + \left\{ \begin{matrix} 8 \text{ bit deslocamento} \\ 16 \text{ bit deslocamento} \end{matrix} \right\}$$
$$PA = \left\{ \begin{matrix} DS \\ SS \end{matrix} \right\} \times 16 + \left\{ \begin{matrix} (BX) \\ (BP) \end{matrix} \right\} + \left\{ \begin{matrix} (DI) \\ (SI) \end{matrix} \right\} + \left\{ \begin{matrix} 8 \text{ bit deslocamento} \\ 16 \text{ bit deslocamento} \end{matrix} \right\}$$

- Usado para acessar matrizes bidimensionais ou matrizes contidas em estruturas.
- `MOV ARRAY[BX][DI],AX`

ARQUITETURA 64 bits

- Registrador apontador de instruções:

REG. 16 BITS	PROPÓSITO
IP	APONTADOR DE INSTRUÇÕES

Aponta sempre para próxima instrução que será executada. É incrementado da quantidade de bytes da instrução que foi previamente executada.



Acesando Matrizes

■ Matrizes unidimensionais.

- `MOV ARRAY[SI*SF],DX`
- SF = fator de escala para o tamanho dos dados.

■ Matrizes bidimensionais.

- `MOV ARRAY[BX*SF*SR][SI*SF],DX`
- SF = fator de escala para o tamanho dos dados.
- SR = tamanho da linha.

Acessando Matrices

Assume the following array definition:

```
ARRAY    DD    00112233H, 44556677H, 88990011H
```

Begin:

```
    LEA BX,ARRAY
```

L1:

```
    MOV AX,[BX]
```

```
    INC BX
```

```
    JMP L1
```

Begin:

```
    MOV SI,0
```

L1:

```
    MOV AX,ARRAY[SI]
```

```
    INC SI
```

```
    JMP L1
```

Begin:

```
    MOV SI,0
```

L1:

```
    MOV AX,DS:ARRAY[SI*4]
```

```
    INC SI
```

```
    JMP L1
```



Alinhamento

- É melhor para alinhar palavras com endereços de número par, e palavras duplas (double word) para endereços divisíveis por quatro, mas isto não é necessário.
- O alinhamento permite o acesso à memória de forma mais eficiente, mas é menos flexível.

Imediato Memória

- Para ler ou escrever na memória usando modo de endereçamento imediato, o programador deve especificar o tamanho do dado, caso contrário o montador adotará como default o maior tamanho de dado possível que o processador pode manusear.
- Usar as seguintes diretivas:
 - ☐ Byte ptr.
 - ☐ Word ptr.
 - ☐ Dword ptr.
- `MOV BYTE PTR VAR,2H`

Definição de Segmentos

- A CPU tem vários registradores de segmento:
 - CS (segmento de código).
 - SS (segmento de pilha).
 - DS (segmento de dados).
 - ES (segmento extra).
 - FS, GS (segmentos suplementares disponíveis nos 386s, 486s e Pentiums).
- Toda instrução e diretiva devem obrigatoriamente corresponder a um segmento.
- Normalmente um programa consiste de três segmentos: pilha, dados e código.

Definição de Segmento

- **Definição Model**

- **.MODEL SMALL**

- ☐ Modelo de memória largamente usado.
- ☐ O código deve caber em 64kb.
- ☐ Os dados devem caber em 64kb.

- **.MODEL MEDIUM**

- ☐ O código pode exceder 64kb.
- ☐ Os dados devem caber em 64kb.

- **.MODEL COMPACT**

- ☐ O código deve caber em 64kb.
- ☐ Os dados podem exceder 64kb.

- **MEDIUM e COMPACT são opostos.**

Segment Definition

■ .MODEL LARGE

- ☐ Código e dados podem exceder 64kb.
- ☐ Conjunto simples de dados pode exceder 64kb.

■ .MODEL HUGE

- ☐ Código e dados podem exceder 64kb.
- ☐ Conjunto simples de dados pode exceder 64kb.

■ .MODEL TINY

- ☐ Usado com programas .COM.
- ☐ Código e dados devem caber em um simples segmento de 64kb.

Definição de Segmento

- **Formatos:**

- ☐ Definição simplificada de segmento.
- ☐ Definição completa de segmento.

- **A definição simplificada de segmentos usa as seguintes diretivas para definir segmentos:**

- ☐ **.STACK**
- ☐ **.DATA**
- ☐ **.CODE**

Essas diretivas marcam o início dos segmentos que elas representam.

Definição de Segmento

- A definição completa de segmento usa as seguintes diretivas para definir segmentos:

- Label SEGMENT [opções]

- ;declarações pertencentes ao segmento

- Label ENDS

- O nome do label deve seguir as regras de nomenclatura mostradas.

Definição de Segmento

;SIMPLIFIED SEGMENT DEFINITION

;FULL SEGMENT DEFINITION

.MODEL SMALL

.STACK 64

.DATA

N1 DW 1432H
N2 DW 4365H
SUM DW 0H

.CODE

BEGIN

PROC FAR

MOV AX,@DATA
MOV DS,AX
MOV AX,N1
ADD AX,N2
MOV SUM,AX
MOV AH,4CH
INT 21H

BEGIN

ENDP

END BEGIN

STSEG SEGMENT
DB 64 DUP(?)
STSEG ENDS

DTSEG SEGMENT
N1 DW 1432H
N2 DW 4365H
SUM DW 0H
DTSEG ENDS

CDSEG SEGMENT

BEGIN

PROC FAR

ASSUME CS:CDSEG,DS:DTSEG,SS:STSEG
MOV AX,DTSEG
MOV DS,AX
MOV AX,N1
ADD AX,N2
MOV SUM,AX
MOV AH,4CH
INT 21H

BEGIN

ENDP

CDSEG

ENDS

END BEGIN



Término de Programa

- **Com PC:**

- **MOV AH,4CH**

- INT 21H**

- **Sempre retorna o controle para o SO.**

Transferências Incondicionais

- **JMP**
- **CALL**
- **RET**
- **Estas instruções modificam o registrador IP para ser:**
 - ☐ **Deslocamento seguindo a instrução (label), no caso de JMP e CALL;**
 - ☐ **O endereço armazenado na pilha pela instrução CALL, no caso de RET.**
- **Ex:**
 - ☐ **JMP Again**
 - ☐ **CALL Delay**
 - ☐ **RET**

Desvios Condicionais

- Usadas com inteiros não sinalizados
 - ☐ JA/JNBE – Desvie se acima
 - ☐ JAE/JNB – Desvie se acima ou igual
 - ☐ JB/JNA – Desvie se abaixo
 - ☐ JBE/JNA – Desvie se abaixo ou igual
- Usadas com inteiros sinalizados
 - ☐ JG/JNLE – Desvie se maior
 - ☐ JGE/JNL – Desvie se maior ou igual
 - ☐ JL/JNGE – Desvie se menor
 - ☐ JLE/JNG – Desvie se menor ou igual
- Outras tipos de desvios
 - ☐ JE/JZ – Desvie se igual
 - ☐ JNE/JNZ – Desvie se não for igual
 - ☐ JC – Desvie se carry
 - ☐ JNC – Desvie se não ocorrer carry
 - ☐ JS – Desvie se sinal
 - ☐ JNS – Desvie se não sinal

Desvios Condicionais

- ☐ JO – Desvie se overflow
- ☐ JNO – Desvie se não ocorreu overflow
- ☐ JP/JPE – Desvie se paridade/paridade par
- ☐ JNP/JPO – Desvie se não paridade/paridade ímpar
- **Essas instruções condicionais modificam o registrador IP para se um dos dois endereços definidos como segue:**
 - ☐ Um endereço ou deslocamento após a instrução (label);
 - ☐ O endereço da instrução após o desvio condicional.
- **Ex:**
 - ☐

```
JE SUM
SUB EAX,EBX
SUM:
```

Controle de Iterações

- **LOOP**
- **LOOPE/LOOPZ**
- **LOOPNE/LOOPNZ**
- **As instruções acima listadas são usadas condicionalmente e incondicionalmente para controlar o número de iterações de um programa através do laço.**
- **Operação do LOOP:**
 - $CX \leftarrow CX - 1$
 - **If $CX \neq 0$**
 then $IP \leftarrow IP + \text{deslocamento}$
 - **Não afeta Flags.**

Controle de iterações

■ Ex:

- ☐ `MOV CX,2`
- ☐ Again: `NOP`
- ☐ `LOOP Again`

■ O que acontece se:

`MOV CX,2`

é substituído por

`MOV CX,0`

no código dado acima.

Controle de iterações

■ Operação de LOOPE/LOOPZ:

- $CX \leftarrow CX - 1$

- If $ZF = 1$ e $CX \neq 0$

 - then $IP \leftarrow IP + \text{deslocamento}$

- Não afeta flags.

■ Operação de LOOPNE/LOOPNZ:

- $CX \leftarrow CX - 1$

- If $ZF = 0$ e $ECX \neq 0$

 - then $IP \leftarrow IP + \text{deslocamento}$

- Não afeta flags.

■ Note que outras instruções dentro do laço podem modificar a condição de ZF.

Controle de iterações

■ Ex:

- ☐ MOV CX,9
- ☐ MOV SI, -1
- ☐ MOV AL, 'D'
- ☐ Again: INC SI
- ☐ CMP AL, LIST[DI]
- ☐ LOOP NE Again
- ☐ JNZ NOT_FOUND

- **JECXZ/JCXZ** – Instruções de desvio condicional se o registrador ECX/CX for igual a zero. São usadas antes de uma instrução do laço para assegurar que o contador de interação , valor em CX não seja zero

ARQUITETURA 64 bits

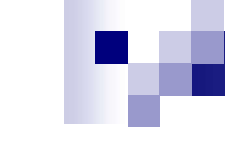
- Registrador apontador de instruções:

REG. 16 BITS	PROPÓSITO
IP	APONTADOR DE INSTRUÇÕES

Aponta sempre para próxima instrução que será executada. É incrementado da quantidade de bytes da instrução que foi previamente executada.

Interrupts

- **INT**
- **INTO – Interrupt if overflow**
- **IRET**
- **These instructions modify the EIP register to be the address stored at:**
 - **The IDT. The interrupt type or number is used to identify which element of the IDT holds the addresses of the desired interrupt service subroutines;**
 - **The stack. The address stored in the stack by the INT or INTO instruction. This address identifies the return point after the interrupts execution.**



Passing Arguments To Subroutines or Modules

■ Via Registers.

- Number of registers is a major limitation associated with this method.
- It is important to clearly document registers used.

■ Via Memory.

- Used by DOS and BIOS.
- Difficult standardization.
- Defined area of RAM is used to pass arguments.



Passing Arguments To Subroutines or Modules

■ Via Stack.

- ☐ Most widely used method of passing parameters.
- ☐ Register and memory independent.
- ☐ Need to be thoroughly understood due to the fact that the stack is used by both the system and the user, so if the stack gets compromised the program can crash.



String Instructions

- **String instructions were designed to operate on large data structures.**
- **The SI and DI registers are used as pointers to the data structures being accessed or manipulated.**
- **The operation of the dedicated registers stated above are used to simplify code and minimize its size.**

String Instructions

- The registers(DI,SI) are automatically incremented or decremented depending on the value of the direction flag:
 - DF=0, increment SI, DI.
 - DF=1, decrement SI, DI.
- To set or clear the direction flag one should use the following instructions:
 - CLD to clear the DF.



String Instructions

- **The REP/REPZ/REPNZ prefixes are used to repeat the operation it precedes.**
- **String instructions we will discuss:**
 - ☐ **LODS**
 - ☐ **STOS**
 - ☐ **MOVS**
 - ☐ **CMPS**
 - ☐ **SCAS**



LODS/LODSB/ LODSW/LODSD

- **Loads the AL, AX or EAX registers with the content of the memory byte, word or double word pointed to by SI relative to DS. After the transfer is made, the SI register is automatically updated as follows:**
 - ☐ **SI is incremented if DF=0.**
 - ☐ **SI is decremented if DF=1.**

LODS/LODSB/ LODSW/LODSD

■ Examples:

- **LODSB**

AL=DS:[SI]; SI=SI ± 1

- **LODSW**

AX=DS:[SI]; SI=SI ± 2

- **LODSD**

EAX=DS:[SI]; SI=SI ± 4

- **LODS MEAN**

AL=DS:[SI]; SI=SI ± 1 (if MEAN is a byte)

- **LODS LIST**

AX=DS:[SI]; SI=SI ± 2 (if LIST is a word)

- **LODS MAX**

EAX=DS:[SI]; SI=SI ± 4 (if MAX is a double word)

LODS/LODSB/ LODSW/LODSD

Example

Assume:

Location	Content
Register SI	500H
Memory location 500H	'A'
Register AL	'2'

After execution of LODSB

If DF=0 then:

Location	Content
Register SI	501H
Memory location 500H	'A'
Register AL	'A'

Else if DF=1 then:

Location	Content
Register SI	4FFH
Memory location 500H	'A'
Register AL	'A'



STOS/STOSB/ STOSW/STOSD

- **Transfers the contents of the AL, AX or EAX registers to the memory byte, word or double word pointed to by DI relative to ES. After the transfer is made, the DI register is automatically updated as follows:**
 - ☐ **DI é incrementado se DF=0.**
 - ☐ **DI é decrementado se DF=1.**

STOS/STOSB/ STOSW/STOSD

■ Examples:

- STOSB

ES:[DI]=AL; DI=DI $\pm$ 1

- STOSW

ES:[DI]=AX; DI=DI $\pm$ 2

- STOSD

ES:[DI]=EAX; DI=DI $\pm$ 4

- STOS MEAN

ES:[DI]=AL; DI=DI $\pm$ 1 (if MEAN is a byte)

- STOS LIST

ES:[DI]=AX; DI=DI $\pm$ 2 (if LIST is a word)

- STOS MAX

ES:[DI]=EAX; DI=DI $\pm$ 4 (if MAX is a double word)

STOS/STOSB/ STOSW/STOSD

Example

Assume:

Location	Content
Register DI	500H
Memory location 500H	'A'
Register AL	'2'

After execution of STOSB

If DF=0 then:

Location	Content
Register DI	501H
Memory location 500H	'2'
Register AL	'2'

Else if DF=1 then:

Location	Content
Register DI	4FFH
Memory location 500H	'2'
Register AL	'2'

MOVS/MOVSb/ MOVSW/MOVSd

- **Transfere o conteúdo de um byte, word ou double word apontado por SI relativo a DS para um byte, word ou double word apontado por DI relativo a ES. Após a transferência ser feita os registradores SI e DI são automaticamente atualizados como segue:**
 - **SI e DI são incrementados se DF=0.**
 - **SI e DI são decrementados se DF=1.**

MOVS/MOVS_B/ MOVSW/MOVSD

■ Examples:

- MOVSB

ES:[DI]=DS:[SI]; DI=DI ± 1; SI=SI ± 1

- MOVSW

ES:[DI]= DS:[SI]; DI=DI ± 2; SI=SI ± 2

- MOVSD

ES:[DI]=DS:[SI]; DI=DI ± 4; SI=SI ± 4

- MOVS MEAN

ES:[DI]=DS:[SI]; DI=DI ± 1; SI=SI ± 1 (if MEAN is a byte)

- MOVS LIST

ES:[DI]=DS:[SI]; DI=DI ± 2; SI=SI ± 2 (if LIST is a word)

- MOVS MAX

ES:[DI]=DS:[SI]; DI=DI ± 4; SI=SI ± 4 (if MAX is a double word)

MOVS/MOVSB/ MOVSW/MOVSD

Example

Assume:

Location	Content
Register SI	500H
Register DI	600H
Memory location 500H	'2'
Memory location 600H	'W'

After execution of MOVSB

If DF=0 then:

Location	Content
Register SI	501H
Register DI	601H
Memory location 500H	'2'
Memory location 600H	'2'

Else if DF=1 then:

Location	Content
Register SI	4FFH
Register DI	5FFH
Memory location 500H	'2'
Memory location 600H	'2'

CMPS/CMPSB/ CMPSW/CMPSD

- **Compares the contents of the the memory byte, word or double word pointed to by SI relative to DS to the memory byte, word or double word pointed to by DI relative to ES and changes the flags accordingly. After the comparison is made, the DI and SI registers are automatically updated as follows:**
 - **DI and SI are incremented if DF=0.**
 - **DI and SI are decremented if DF=1.**

SCAS/SCASB/ SCASW/SCASD

- **Compares the contents of the AL, AX or EAX register with the memory byte, word or double word pointed to by DI relative to ES and changes the flags accordingly. After the comparison is made, the DI register is automatically updated as follows:**
 - **DI is incremented if DF=0.**
 - **DI is decremented if DF=1.**

REP/REPZ/REPNZ

- These prefixes cause the string instruction that follows them to be repeated the number of times in the count register ECX or until:
 - ZF=0 in the case of REPZ (repeat while equal).
 - ZF=1 in the case of REPNZ (repeat while not equal).



REP/REPZ/REPNZ

- **Use REPNE and SCASB to search for the character 'f' in the buffer given below.**
- **BUFFER DB 'EE3751'**
- **MOV AL,'f'**
- **LEA DI,BUFFER**
- **MOV ECX,6**
- **CLD**



REP/REPZ/REPNZ

- Use REPNE and SCASB to search for the character '3' in the buffer given below.
- BUFFER DB 'EE3751'
- MOV AL,'f'
- LEA DI,BUFFER
- MOV ECX,6
- CLD
- REPNE SCASB



80386

- **General purpose processor optimized for multitasking operating systems.**
- **Supports 32 bits address and data buses.**
- **Capable of addressing 4 gigabytes of physical memory and 64 terabytes of virtual memory.**

Registers

■ General purpose registers.

- There are eight 32 bits registers (EAX, EBX, ECX, EDX, EBP, EDI, ESI, and ESP).
- They are used to hold operands for logical and arithmetic operations and to hold addresses.
- Access may be done in 8, 16 or 32 bits.
- There is no direct access to the upper 16 bits of the 32 bits registers.
- Some instructions incorporate dedicated registers in their operations which allows for decreased code size, but it also restricts the use of the register set.



Registers

■ Segment registers.

- There are six 16 bits registers (CS, DS, ES, FS, GS, and SS).
- They are used to hold the segment selector.
- Each segment register is associated with a particular kind of memory access.



Registers

■ Other registers.

- EFLAGS controls certain operations and indicates the status of the 80836 (carry, sign, etc).
- EIP contains the address of the next instruction to be executed.
- The E prefix in all 32 bits registers names stands for extended.

80386 Architecture

EAX		AX	Accumulator
		AH AL	
EBX		BX	Base Index
		BH BL	
ECX		CX	Count
		CH CL	
EDX		DX	Data
		DH DL	
EDI		DI	Destination Index
ESI		SI	Source Index
ESP		SP	Stack Pointer
EBP		BP	Base Pointer
EIP		IP	Instruction Pointer
EFLAGS		FLAGS	Flags
		CS	Code
		DS	Data
		ES	Extra
		SS	Stack
		FS	Supplemental
		GS	

Effective, Segment and Physical Addresses

- **Effective address (EA).**

- ☐ Also called offset.
- ☐ Result of an address computation.

- **Segment address (SA).**

- ☐ Also called segment selectors.
- ☐ Addresses stores in segment registers

- **Physical address (PA).**

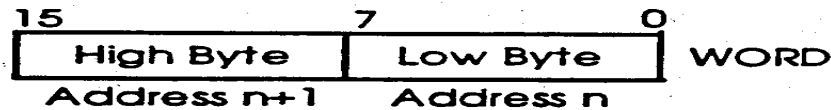
- ☐ Location in memory.
- ☐ $PA = SA * 16 + EA$

Memory Organization

- Sequence of bytes each with a unique physical address.
- Data types:

- ☐ Byte.

- ☐ Word.



Little Endian Notation

- The 80386 stores the least significant byte of a word or double word in the memory location with the lower address.

Assuming EAX = 11223344H

mov ds:[500H], EAX

500H	⋮
	44H
	33H
	22H
	11H
	⋮

PC Parallel Printer Port

■ Types:

- SPP – Standard Printer Port
- PS/2 – Simple bidirectional
- EPP – Enhanced Parallel Port
- ECP – Extended Capabilities Port

■ Addressing:

□ Base addresses:

- 278H
- 378H
- 3BCH

■ Registers:

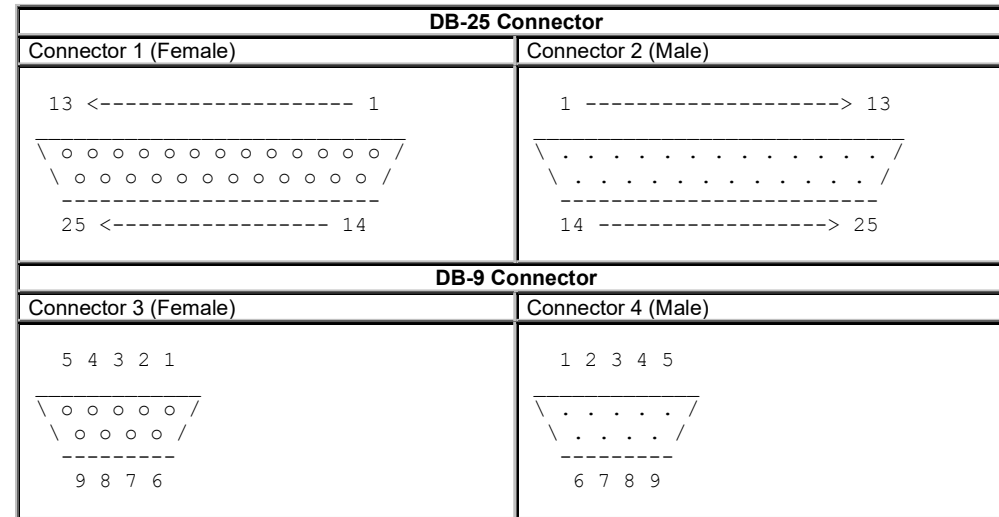
- Data, 8 bits, base address

PC Parallel Printer Port

Data Register (Base Address)				
Bit	Pin: DB-25	Signal Name	Inverted at connector?	I/O
0	2	Data bit 0	No	Output
1	3	Data bit 1	No	Output
2	4	Data bit 2	No	Output
3	5	Data bit 3	No	Output
4	6	Data bit 4	No	Output
5	7	Data bit 5	No	Output
6	8	Data bit 6	No	Output
7	9	Data bit 7	No	Output
Status Register (Base Address + 1)				
Bit	Pin: DB-25	Signal Name	Inverted at connector?	I/O
3	15	nError	No	Input
4	13	Select	No	Input
5	12	PaperEnd	No	Input
6	10	nAck	No	Input
7	11	Busy	Yes	Input
Control Register (Base Address + 2)				
Bit	Pin: DB-25	Signal Name	Inverted at connector?	I/O
0	1	NStrobe	Yes	Output
1	14	nAutoLF	Yes	Output
2	16	Ninit	No	Output
3	17	nSelectIn	Yes	Output
4		IRQ 1 = enabled		
5		Bidirectional 1 = input		

DB-25 and DB-9 Pin Diagram

The "o" represent holes, the "." represent pins.

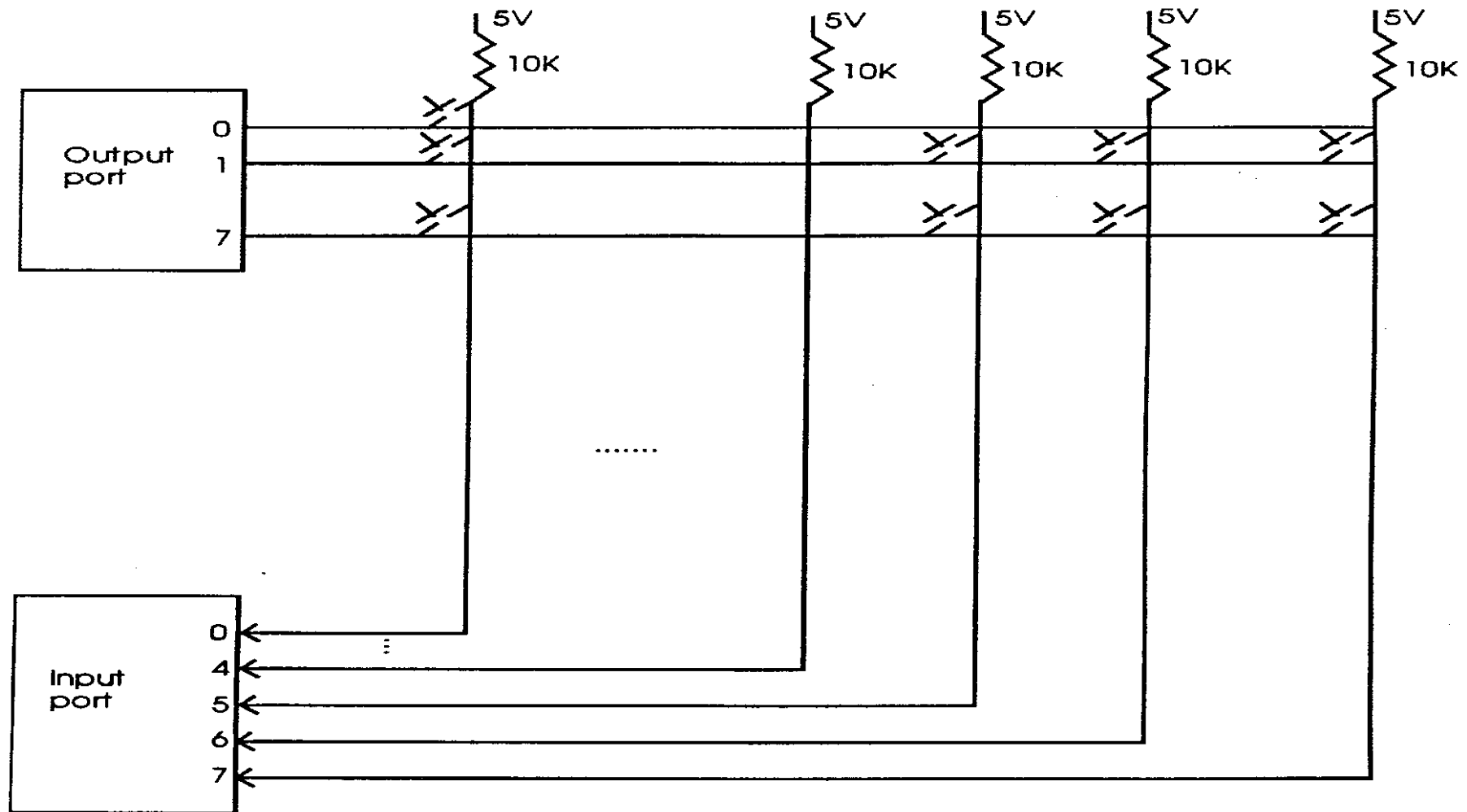


Each diagram shown above is the view you see when you look into the end of the cable.

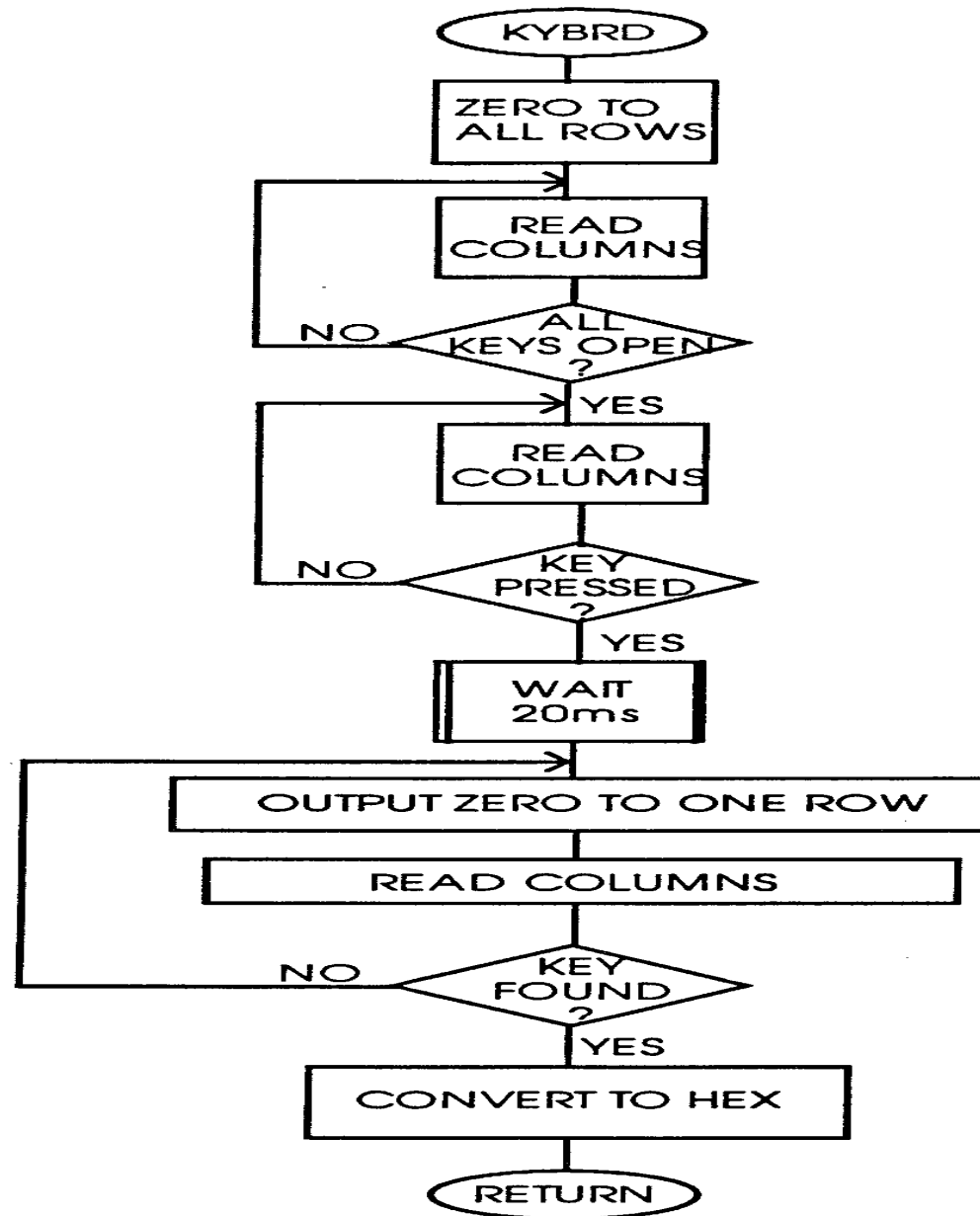
Keyboard Interfacing

- **There are several types of keyboards available for computer usage. Some of the most common types are:**
 - ☐ Mechanical switches
 - ☐ Membrane switches
 - ☐ Capacitive switches
 - ☐ Hall effect key switches
- **Most keyboards are organized as a matrix of rows and columns. Getting data from the keyboard requires the following steps:**
 - ☐ Detect a key press.
 - ☐ Debounce the key press.

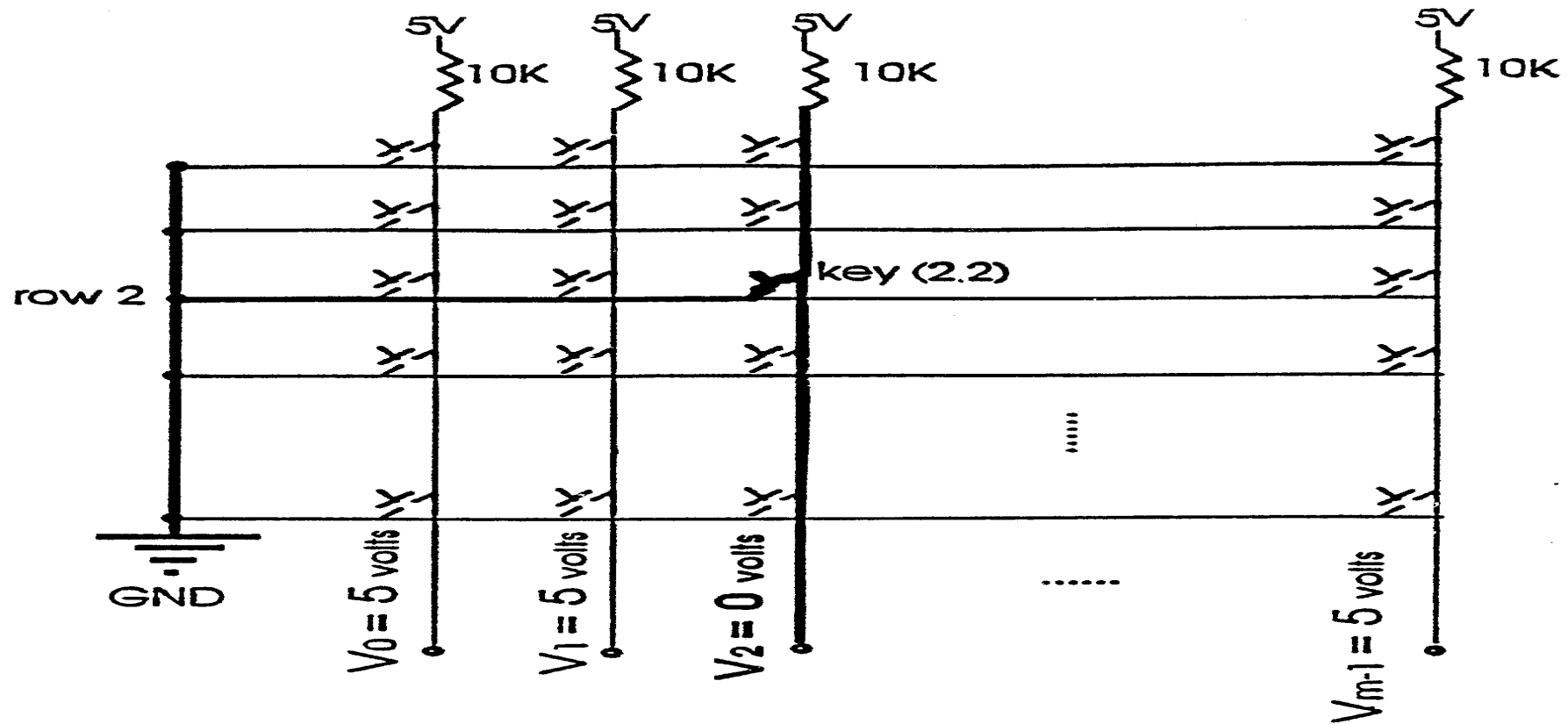
Keyboard Interfacing



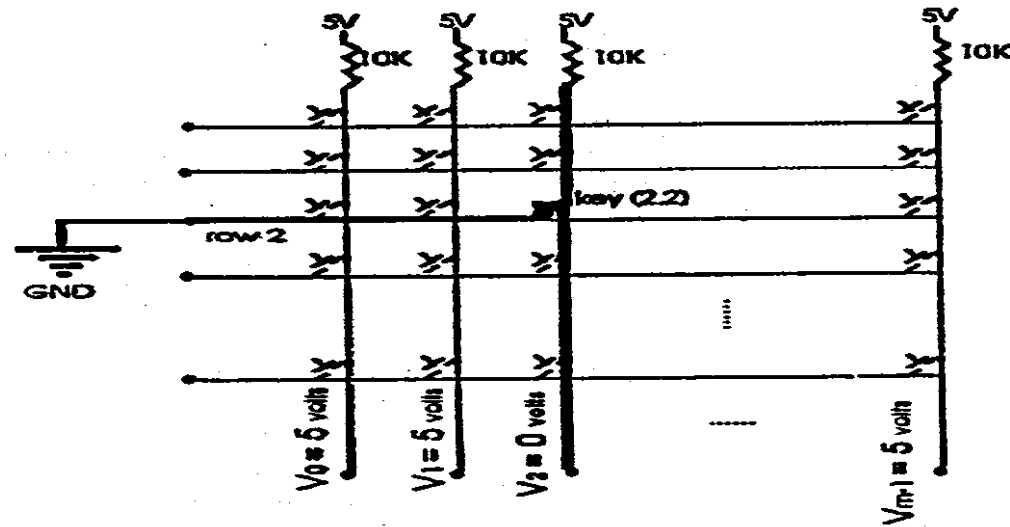
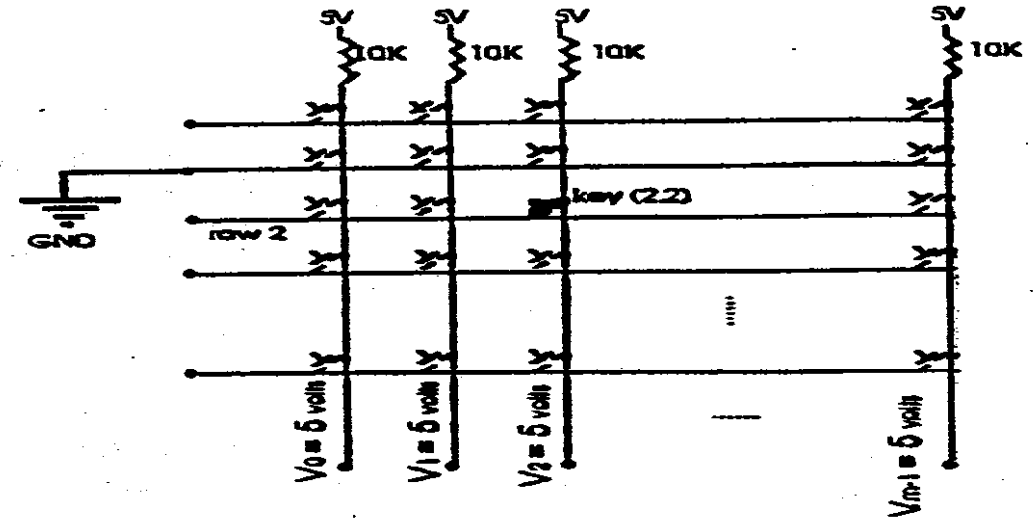
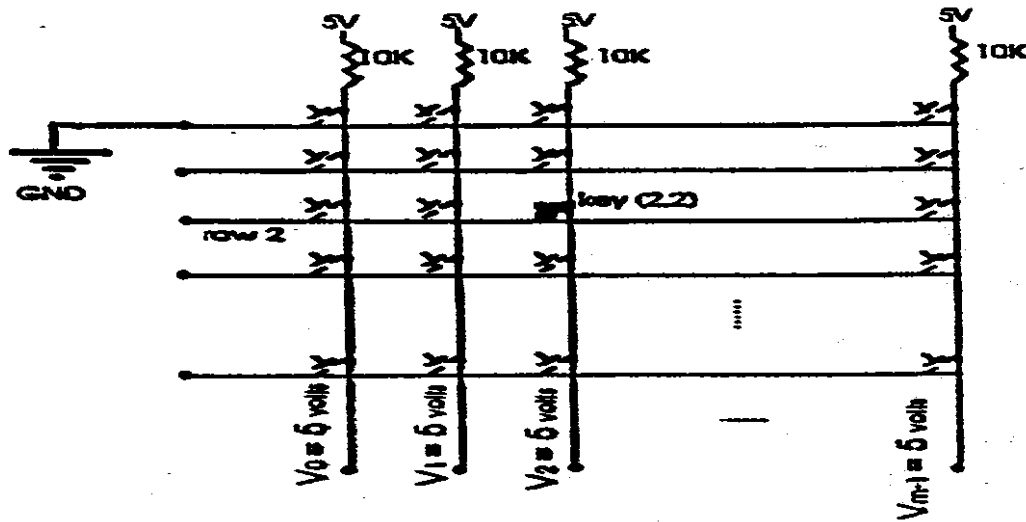
Keyboard Interfacing



Keyboard Interfacing



Keyboard Interfacing



Keyboard Interfacing

- **Encoding the key press:**
 - Find the row and column positions (obtained from the key detection routine).
 - Calculate the offset using the following formula:
$$\text{OFFSET} = (\text{row} * 8) + \text{column}$$
 - 8 is the number of columns in the keyboard matrix.
 - Find the proper character using the offset, the base address of the conversion table and XLATB

Interrupts

- **Interrupts/exceptions are actions prompting the transfer of program execution to some special routine.**
- **Interrupt/exception Service Routine is the routine executed as a result of an interrupt/exception call.**
- **Interrupts:**
 - **Maskable Interrupts (MI):**
 - Do not occur unless interrupt flag is set.
 - STI – sets interrupt flag.
 - CLI – clears interrupt flag.
 - **Non-Maskable Interrupt (NMI):**
 - No mechanism is provided to prevent NMI's.



Interrupts

- **Exceptions:**

- ☐ **Some instructions may generate exceptions.**

- Example: DIV may generate the divide by zero exception.**

- **Interrupt Descriptor Table (IDT), also known as Interrupt Vector Table, is a data structure used for the purpose of handling interrupts. They associate each interrupt/exception with an address indicating the location of the Interrupt Service Routine which will be used to service the calling interrupt.**